

大規模言語モデルを用いた 1DCAE のモデリング作業の支援

*¹池田 孟, *²石橋 祐太郎, *¹金丸 正寛, *¹小林 孝

Supporting 1DCAE modeling using Large Language Model

*¹Hajime IKEDA, *²Yutaro ISHIBASHI, *¹Masahito Kanamaru and *¹Takashi KOBAYSHI

*¹三菱電機株式会社 設計開発技術センター Engineering and Development Center, Mitsubishi Electric Corporation

*²三菱電機株式会社 生産技術センター Manufacturing Technology Center, Mitsubishi Electric Corporation
8-1-1, Tsukaguchi-Honmachi, Amagasaki City, Hyogo 661-8661, Japan

Abstract

In complex system design, where mechanical, electrical, and control subsystems interact, 1D CAE is increasingly adopted to optimize products quickly by verifying requirements through early-stage physical simulations on virtual prototypes. The Modelica language has become the standard for modeling these systems due to its object-oriented, declarative nature and ability to handle multi-domain interactions. It supports hierarchical modeling using extensive libraries. Recent advancements in generative AI, particularly Large Language Models (LLMs), have enabled new approaches in design engineering, such as automating design system handling and managing design knowledge. This study presents a framework that uses Retrieval-Augmented Generation (RAG) to search closed corporate Modelica repositories and employs three collaborative agents—a planner, coder, and tester—to automatically assemble libraries. Experiments using OpenAI-4o demonstrated that this agent-based workflow could generate candidate code for tasks that RAG alone could not handle. The results also showed improved code generation performance when working with proprietary internal libraries, highlighting the potential of combining LLMs with structured agent collaboration in engineering design automation.

Keywords : Large Language Model, Model-based Development, 1DCAE, Design Engineering

1. 緒 言

近年、機械・電気・制御といった複数のサブシステムが複雑に連携するシステム設計において、設計初期の段階からシミュレーションを活用し、仮想プロトタイプ上で要求仕様の充足性を検証しながら、製品の短期最適化を図る「1DCAE（一次元 CAE）」の手法が広く用いられるようになっている(日本機械学会, 2020)．物理シミュレーションモデルの記述には、オブジェクト指向かつ宣言型で多領域連成を自然に扱える Modelica 言語が事実上の標準として位置づけられている．Modelica Standard Library をはじめ、熱流体、電磁気、制御系など様々なドメインに対応したサブモデルがライブラリとして公開されており、ユーザは既存のコンポーネントを階層的に組み合わせモデルを構築できる．設計工学分野では、設計知識の体系化や設計システムの自動化に生成 AI の活用が試みられている(藤田喜久雄, 2023)．に大規模言語モデル(LLM)の急速な進化によって、設計支援における活用範囲が拡大しつつある．本研究では、企業内でクローズドに運用されている 1DCAE ライブラリに対して、LLM を活用したモデリング作業・読解支援を試みた．具体的には、RAG (Retrieval-Augmented Generation) を用いた関連ドキュメント検索と、エージェントによる協調作業によってライブラリ自動組立を行うフレームワークを構築した．OpenAI 4o-mini を用いた実験では、RAG 単体では対応できなかったコード生成タスクについて、エージェント型ワークフローにより候補コードの生成が可能となった．企業内のクローズドなライブラリ活用においても、エージェントワークフローによるコード生成能力の向上が確認された．

2. 設計における 1DCAE の役割

設計現場において 1DCAE は、単なるシミュレーション手段にとどまらず、設計ノウハウの集約や設計根拠の体系的な蓄積・管理といった役割も担っている((藤田喜久雄, 2023)). ここでは、設計における 1DCAE のモデルとライブラリの意義について整理する。

2.1 コンカレントエンジニアリングを実現するための 1DCAE

現代の製品開発では、メカの挙動をソフトウェア制御によって細かく最適化することが重要な価値創出要素となり、複数領域を横断する設計が不可欠となっている。このため、メカ・電気・ソフトウェア（制御）など異なる領域の設計工程を重ね合わせて並行して進める「コンカレントエンジニアリング」が多くの企業で実践されている。こうした領域横断型設計を効率的に進めるため、シミュレーションモデルの活用によるモデルベース開発 (MBD) が注目されている。

MBD においては、メカ領域は 3 次元 CAD ベースの CAE により挙動・特性の解析が行われる一方、電気や制御領域では 1 次元の集中定数系シミュレーションが多用される。メカ領域の 3 次元モデルを抽象化し、他領域のシミュレーションと連携可能にしたモデルを「1D モデル」と呼び、そのシミュレーションツールが「1DCAE」である。1DCAE は MBD の根幹を成すシミュレーション技術として重要な役割を担っている。

2.2 設計カタログとしての 1DCAE ライブラリ

設計工学では、効率的な設計活動のための物理法則や経験則を「設計知識」と呼ぶ。これらを体系的にまとめたものが「設計カタログ」であり、設計根拠や知識の再利用性向上に寄与する。1DCAE は制御シミュレーションや物理シミュレーションのモデル群をライブラリとして集約しており、理論や実験データが蓄積されている。これによりモデルの忠実度 (fidelity) を裏付け、ライブラリは利用実務とともに発展・拡張し続ける。完成度の高い 1DCAE ライブラリは、物理法則やその根拠となるデータを備え、設計カタログとしての機能を果たす。開発者はライブラリを活用することで、既存モデルを基に短期間で新規モデルを構築できる。すなわち、1DCAE におけるモデルは、単なるシミュレーション手段ではなく、設計根拠のカタログとして機能している。

2.3 1DCAE のライブラリを活用における実務課題

このようなモデリング作業には、モデル対象への深い理解、プログラム言語と専門ライブラリの仕様、数値計算の知識や経験など、高度なドメイン知識と技能が求められる。そのため、モデリング・シミュレーションを実践可能なエンジニアの育成には多大な時間とコストを要する。物理シミュレーション言語 Modelica や数値流体計算ライブラリ OpenFOAM などは、研究開発現場で広く使われているが、ライブラリの複雑さやモデル対象の専門性から、高度な専門知識を持つエンジニア・研究者が不可欠となる。専門分野が狭いため、一般書籍やレビュー誌など体系的な情報が少なく、ライブラリ学習の障壁となっている。シミュレーション手段と設計カタログの双方の特性を持つライブラリ活用には、依然として高いスキルが要求される。

3. 先行研究

3.1 LLM の設計の展開事例

2024 年以降に報告された最近の設計問題の LLM を適用した事例を紹介する。現在では、設計知識の管理、作業の自動化、情報の共有といった設計問題に LLM が使われるようになってきている。Picard ら (Picard et al., 2024) は、概念設計から製造・検査に至る四つの設計フェーズを対象として、マルチモーダル Vision-Language Model (VLM) である GPT-5V と LLaVA 1.6 34B の有効性を体系的に評価した。同研究は VLM が生成した回答根拠を設計図や仕様書にトレースする手法を提案し、設計者が結果を検証しやすい解釈性向上の方向性を示した。多くの設計知識は、構造データではなく、非構造データである。LLM は設計における非構造データの認識処理に有効であることを示している。トヨタ自動車は、熟練エンジニアの暗黙知を LLM にリンクさせて次世代車両の開発速度を高める目的で、マルチエージェント型の設計支援基盤を構築している (Onishi, 2024)。同社では、パワートレイン設計部門において LLM で構成されたエージェントが設計資料の探索を加速する事例が報告された。これらのエージェントは、制御シミュレーションや法規制などの知識を検索し、設計根拠の探索と解釈を支援している。三菱電機は、設計・生産準備を含む ECM (Engineering Chain Management) 領域に生成 AI を組み

込む社内プラットフォームを構築し、2017 年以降の AI 活用技術を集約したソリューション群を展開している(玉谷基亮, 2024)。同社は技術文書のレイアウト解析と差分照合を行う自然言語処理 AI、FMEA/DRBFM データを参照して新規設計変更リスクを自動抽出するリスク抽出 AI、CAE 解析結果を学習したサロゲートモデル AIなどを設計フローに導入し、設計パラメーター探索の高速化と知見の再利用を図った。加えて、デジタルツインと生成 AI エージェントを連携させ、IoT で収集した実機データと仮想空間のシミュレーション結果を横断的に解析する設計最適化の将来像を提案している。これら先行研究では、クローズドな社内情報を RAG など検索システムで設計活動に活用する事例が増えており、設計知識管理や情報共有への LLM 活用が進展している。

3.2 LLM による専門ライブラリの活用支援

本研究は Modelica 言語によるライブラリのモデリング作業支援を主題とする。Modelica に限らず、物理シミュレーション分野では既存ライブラリ操作が中心となる。近年の大規模言語モデル LLM はコード生成力の向上により、Modelica のコンポーネント自動生成への応用が進んでいる。Xiang(Xiang et al., 2025)は、Modelica のコンポーネントを作成する LLM エージェント「ModiGen」を構築した。近年、LLM のコード生成能力向上に伴い、Modelica コンポーネント自動生成への応用事例が増えている。Xiang ら(2025)は、Modelica コンポーネントを自動生成する LLM エージェント「ModiGen」を開発し、ベンチマーク問題に対し 1 回の指示で一定の完成度を持つコード生成が可能であることを報告した。ベンチマーク問題のコードの自動化の成功率は 30% から 50%であった。

数値流体シミュレーションでよく使われる OpenFOAM は、C++言語で作成されたライブラリである。Pandy (P et al.,)は、OpenFOAM を用いた CFD シミュレーションの自動化を目的として、GPT-4o および CoT 機能搭載モデルを基盤とするエージェント「OpenFOAMGPT」が提案している。本手法では、OpenFOAMチュートリアルケースから得た領域知識を検索拡張生成(RAG)パイプラインに組み込み、入力ファイルの自動生成・実行・エラー訂正を反復的に行う構造を採用している。評価では、RAG を用いた少数ショットプロンプトにより、境界条件やメッシュ解像度の変更に対応できることが示された。一方、精度担保のための人手監視やモデル性能の時間変動が課題として指摘されている。Chen(Chen et al., 2024),(Chen et al., 2025)は、ソフトウェア開発エージェント MetaGPT を OpenFOAM に適用した「MetaOpenFOAM」は、Architect, InputWriter, Runner, Reviewer の四つのエージェントが連携し、ユーザーの自然言語要求から入力ファイル生成、シミュレーション実行、エラー分析・修正までを自動化する反復ループを構築する。

これらの先行研究では LLM による完全自動化を試行しているが、いまだに LLM によるシミュレーションの完全自動化は難しい。表 1 に完全自動化が困難な理由を列挙する。

Table 1 list of difficulties in applying LLM to simulation modeling

要因	備考
高度なドメイン知識の不足	専門ライブラリは、機密性が高く、インターネットには公開されている活用例は少ない。LLM はインターネット上の広範囲のデータを用いて訓練されているが、Modelica の特定のライブラリなどの情報を汎用的な訓練データから学習することは、難しい。
信頼性と正確性	シミュレーション作業のセットアップは、一般的に厳密な正確性が要求される。モデリング作業は、複数のセットアップ作業を正確に実行する必要があるが、すべての作業を正確に実行することが LLM にとっては、難しい。
ベンチマーク問題の作成	LLM の回答精度を向上させるためには、システムプロンプトの工夫やパラメータのチューニングが必要である。専門ライブラリは、ユーザ数が限られており、専門性の高い知識が必要であり、LLM をチューニングするためのベンチマーク問題を用意することが難しい。

LLM によるシミュレーション完全自動化は困難であるが、専門ライブラリの読解支援やコード開発支援には有効であり、従来専門家のみが扱えたライブラリ活用の敷居を大きく下げる効果が期待できる。

4. 本研究

本研究では、企業における 1DCAE のモデリング作業支援に LLM の適用可能性を検討した。

4.1 本研究で対象とした Modelica ライブラリ

本研究では、空調開発における 1DCAE ライブラリに生成 AI の支援を適用した例を報告する。当社（三菱電機）でクローズされたライブラリの活用に LLM を適用した。

本研究で対象としたライブラリについて説明する。三菱電機では、冷凍サイクルのシミュレーションに米国 Mitsubishi Electric Research Laboratory (MERL) で開発された Modelica ライブラリ「MERL HVAC Library」を利用している。本研究ではこのライブラリを対象とした。MERL HVAC Library は、エアコン等の冷凍サイクルにおいて、熱交換器、圧縮機、膨張弁など各要素の冷媒内部の圧力・温度・比エンタルピーなどを計算可能である。ユーザはライブラリに含まれるモデルを組み合わせることで、再利用性と保守性の高い新規モデルを容易に構築できる。全 8 種類のパッケージから成り、建築熱負荷、冷凍サイクル、制御アルゴリズムに関するモデルが実装されている。冷媒挙動は強い非線形性を持ち、動的シミュレーションが難しいため、物理スケールや時間スケールごとに適切なモデル選択が必要である。MERL HVAC Library では冷媒流の差分化方法や計算スキームの選択も可能である (Laughman(Laughman,)).

シミュレーション時には熱流体計算の複雑さから収束性や計算時間への配慮も重要となる。こうしたノウハウを活かすには、モデルの理論や実験ドキュメントの読解が必須であり、MERL 発表の論文や開発者の博士論文などが参考資料となる。

Table 2 MERL HVAC Library

名称	MERL HVAC Library
開発者	Mitsubishi Electric Research Laboratory (MERL)
機能	冷凍サイクル、建築温熱環境、制御のシミュレーション
パッケージ数	8 種類のパッケージ、ファイル数 3,112 個
関連ドキュメント	公開論文（開発者名義のものが 20 報告）、機密資料

5. LLM による 1DCAE の活用支援

5.1 大規模言語モデルの応用

本稿で構築した 2 つのシステムについて説明する。

5.1.1 RAG システム

図 1 に示す RAG システムは 4 層構造で設計した。

(1) データ層：MERL HVAC Library のコードや関連論文を Markdown 形式に変換し、Sentence-BERT でベクトル化して FAISS へ格納。(2) 検索層：ユーザの質問を同じ埋め込み空間で検索し、上位 k 件を取得。複数チャックを階層的にマージしてプロンプト長を最適化する。(3) 生成層：OpenAI GPT-4o mini をベースとした LLM に対し、役割指示+検索結果+会話履歴で構成されるテンプレートを送信する。(4) 出力層：回答と同時に引用情報を提示し、ユーザはソースコード断片へ直接アクセスできる。一般的な知識で回答可能な場合は LLM 単体の応答を採用する。LLM 単体には「自信がない場合は、その旨を申告する」指示を与えている。

5.1.2 Modelica コード生成エージェント

図2にエージェントのワークフローを示す。エージェント型ワークフローは、Plan-Execute-Critique ループにより「検索→計画→コーディング」を自律実行する点が特徴である。(1) RAG：前述の RAG システムを用いてユーザプロンプトに関連するドキュメントを検索。(2) プランナー：検索結果からモデル構造の手順書(pseudo-plan)を生成。(3) コーダー：手順書に従ってコードスニペットを生成する。

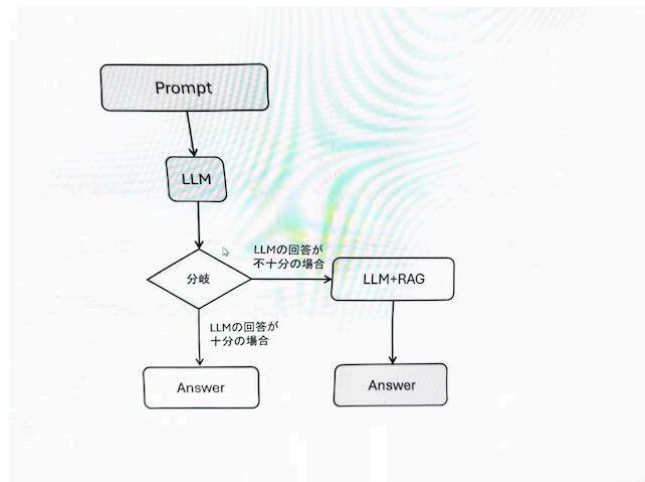


Fig. 1 Workflow chart of our agent.

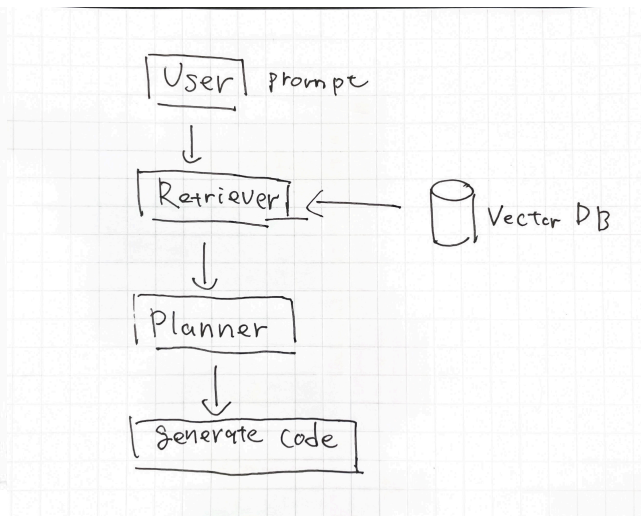


Fig. 2 Workflow chart of our agent.

6. 結果

6.1 RAG システムの実験

6.1.1 一般的な知識で回答できる問題

熱力学などの高度な知識が必要な問いにも、設計エンジニアが参照できる水準で LLM が回答可能であった。表3にプロンプトと回答を記す。

Table 3 Response of LLM

prompt	Explain Helmheltz Energy Equation.
Answer	The Helmholtz free energy F (sometimes written as A) is a thermodynamic potential that measures the maximum useful work obtainable from a closed system at constant temperature and constant volume. — Definition $F = U - TS$ Where: • F or A = Helmholtz free energy [J] • U = Internal energy [J] • T = Absolute temperature [K] • S = Entropy [J/K] — Physical Meaning • It represents the amount of usable energy in a system that can be converted into work, except for the part lost due to entropy.

6.1.2 文書データベースの情報が回答に必要な質問

関連文書データベースの情報が不可欠な質問にも、RAG を介した回答が可能となった。表5にプロンプトと回答を記す。

Table 4 Response of LLM

prompt	Explain the method of refrigerant property calculation. What is Helmholtz Energy Equation? Is that related to refrigerant property?
Answer	The search results explain different methods for calculating refrigerant properties like density, enthalpy, and temperature based on known state variables like pressure and enthalpy. One common approach is to use the chain rule to relate the derivative of density with respect to time to the derivatives of pressure and enthalpy with respect to time (sources 4, 10). The partial derivatives of density with respect to pressure and enthalpy can be calculated analytically from thermodynamic relations (source 2). Source Chunks <pre>{'text': 'On the other hand, it is wise to select the enthalpy as the independent state variable because the enthalpy can be readily used to distinguish whether the refrigerant is in single-phase or two-phase region and hence appropriate correlations for heat transfer and pressure drop can be applied accordingly.'}</pre> <pre>2.2.2 Discretization The partial differential equations (2-1) to (2-3) can be solved by various methods, like finite difference, finite volume and finite element.', 'location': {'s3Location': {'uri': 's3://llmragmerlhvac-hajime-20240909/Qiao_umd_0117E_15712.pdf'}}</pre>

6.1.3 コード作成の指示

ルームエアコン冷凍サイクルモデルの作成を RAG システムに指示した結果を表 5 に示す。Modelica に精通した人間であれば、適切なモデルをライブラリから選択し仕様を確認し、空調分野の知識があれば必要な室内機、室外機、圧縮機、膨張弁のモデルを探し、部屋モデルも含めて動的挙動を再現できるが、LLM 単体では「I don't know.」と応答するに留まった。ただし RAG システムは関連モデルを検索結果として提示できている。

表 5 の出力には、LLM の回答だけでなく、回答の根拠となる検索結果（Source Chunks）を表示している。LLM の回答は、「I don't know.」のみであった。Source Chunks によると、RAG システムは、ルームエアコンに関連するモデルのいくつかは検索結果として取得しているが、その上で無回答を検索している。

Table 5 Response of LLM

prompt	Construct room air conditioner model by MERL-HVAC Library.
Answer	I don't know. Source Chunks: - VCComponents\Components\AirHandling\Testing\MultiportLumpedRoomTest.mo-0 - VCComponents\Components\AirHandling\FanCurveModels\package.mo-0 - VCComponents\Components\AirHandling\FanCurveModels\BaseClasses\package.mo-0 - MEBuildings\USD0E\SmallOffice\FullMode\FourZoneOffice.mo0

6.2 Modelica コード生成エージェントの実験

RAG システムでは対応できなかったコーディング作業についても、エージェント型ワークフローを導入することで、Planner エージェントが実行環境構築やパラメータ設定、モデル接続、制御ロジック実装などの手順を具体的に提示した。表 6 にエージェントワークフローの Planner エージェントの出力を載せる。

Planner は、前段階の RAG エージェントから検索結果を受け取り、ルームエアコンのモデル作成に VCComponents というモデルが関連することを受け取っている。その上で、Modelica の実況環境の構築（'1 Set Up the Environment'）やシミュレーションにいくつかのパラメータ設定が必要なことを確認している（'3 Define Parameters'）。また、'11 Validation:'では、プロンプトの指示外ではあるが、シミュレーション結果の検証を計画した。

Table 6 Response of Planner agent

1 Set Up the Environment:

- Ensure you have a Modelica environment installed (e-g-, OpenModelica, Dymola).
- Import the MERL-IVAC Library (VCComponents) into your Modelica environment.

2 Create a New Model:

- Start a new Modelica model file (e-f*, RoomAirConditioner.mo).
- Define the model structure and include necessary libraries. within YourPackageName; model RoomAirConditioner

3 Define Parameters:

- Define parameters for the room conditions, such as temperature, humidity, and volume. You can use the parameters from the

MultiportLumpedRoomTest model as a reference.

```
parameter SI. Temperature TRoom_init - Modelica. Units. Conversions. from_degC(27.0);
parameter Real RHRRoom_init = 0.51;
parameter SI. Volume volRoom = 87.5 * 77 * 84 * 1.63871e-5 * 10;
```

4 Instantiate the Room Model:

- Use the MultiportLumpedRoomTest model to represent the room. This model will handle the thermal dynamics of the room.

```
MultiportLumpedRoomTest room(TRoom_init TRoom_init, RHRRoom_init-RHRRoom_init, volRoom-volRoom);
```

5 Define the Air Handling System:

- Create an air handling system using components from the FanCurveModels package. You may need to define a fan and a cooling coil.

```
// Example of instantiating a fan and cooling coil
FanCurveModels. Fan fan(...);
CoolingCoil coolingCoil(...);
```

6 Connect Components:

- Connect the room, fan, and cooling coil. Ensure that the air flow from the fan goes into the room and that the cooling coil is properly integrated into the air handling system.connect (fan.outlet, coolingCoil. inlet); connect (coolingCoil.outlet, room.airinlet);

7 Define Control Logic:

- Implement control logic for the air conditioner. This could involve setting temperature setpoints and controlling the fan speed based on room conditions.

```
// Example control logic
if room. TRoom > TRoom setpoint then
fan. speed = maxFanSpeed; // Turn on the fan
else
fan. speed = 0; // Turn off the fan
end if;
```

8 Add HVAC System Record:

- Optionally, you can include the HVACSystem record to manage the operating conditions of the HVAC system.

```
HVACSystem hvacSystem(...);
```

9 Simulation Setup:

- Set up the simulation parameters, including the simulation time and solver settings.

```
simulate(RoomAirConditioner);
```

10 Run the Simulation:

- Execute the simulation and analyze the results, Check the room temperature, humidity, and other relevant outputs to ensure the air conditioner is functioning as expected.

11 Validation:

- Validate the model against known benchmarks or existing models (e-g-, EnergyPlus) to ensure accuracy.

12 Documentation:

- Document the model, including the purpose, parameters, and any assumptions made during the modeling process,

13.Conclusion

By following these steps, you can construct a room air conditioner model using the MERL-HVAC Library (VCComponents). Adjust the parameters and components as necessary to fit your specific requirements and ensure the

表 7 に **Code Generation** エージェントの出力を載せる。実際にはエージェントの出力したコードはそのままでは、コンパイルが通らない。モデルの完成にはパラメータ値の設定が必要になる。ただし、選択したコンポーネントには間違いはなく、モデル作成の初期段階の作業支援の結果としては十分である。**Code Generation** エージェントが **Modelica** コードの雛形を自動生成できることを確認した。これにより、従来専門知識が必要だったモデル構築作業の自動化・省力化が可能となった。

Table 7 Response of Code generation agent.

```

within YourPackageName;

model RoomAirConditioner

// Import necessary libraries
import Modelica.SIunits;
import VCComponents.Components.AirHandling.Testing.MultiportLumpedRoomTest;
import VCComponents.Components.AirHandling.FanCurveModels;
import MEBuildings.USDOE.ResidentialPrototype.BaseClasses.RecordTemplates.OperatingConditions.HVACSystem;

// Define parameters for the room conditions
parameter SI.Temperature TRoom_init = Modelica.Units.Conversions.from_degC(27.0);
parameter Real RHRoom_init = 0.51;
parameter SI.Volume volRoom = 87.5 * 77 * 84 * 1.63871e-5 * 10;

// Instantiate the room model
MultiportLumpedRoomTest room(
    TRoom_init=TRoom_init,
    RHRoom_init=RHRoom_init,
    volRoom=volRoom);

// Define the air handling system components
FanCurveModels.Fan fan(
    // Define fan parameters here
    // Example: flowRate=..., pressureRise=..., etc.
);

CoolingCoil coolingCoil(
    // Define cooling coil parameters here
    // Example: coolingCapacity=..., etc.
);

// Connect components
connect(fan.outlet, coolingCoil.inlet);
connect(coolingCoil.outlet, room.airInlet);

// Define control logic
parameter SI.Temperature TRoom_setpoint = Modelica.Units.Conversions.from_degC(24.0);
parameter Real maxFanSpeed = 1.0; // Maximum fan speed

// Control logic for the air conditioner
equation
    if room.TRoom > TRoom_setpoint then
        fan.speed = maxFanSpeed; // Turn on the fan
    else
        fan.speed = 0; // Turn off the fan
    end if;

// Optionally, include the HVAC system record

HVACSystem hvacSystem(
    // Define HVAC system parameters here
    // Example: eps=..., TAirSupply=..., etc.
);

end RoomAirConditioner;

```

7. 結 言

本稿では、大規模言語モデルによる物理シミュレーション分野への作業支援について検討した。現時点では、実用的なライブラリを用いた複雑なコーディング作業を完全に自動化することは困難であるものの、知識探索やコード作成方針の支援には十分実用的であることを示した。今後、LLM とエージェント技術を組み合わせることで、企業内専門ライブラリを活用した設計自動化・効率化の可能性がさらに広がると考えられる。

References

- Chen, Y., Zhang, L., Zhu, X., Zhou, H. and Ren, Z., OptMetaOpenFOAM: Large Language Model Driven Chain of Thought for Sensitivity Analysis and Parameter Optimization based on CFD (2025).
- Chen, Y., Zhu, X., Zhou, H. and Ren, Z., MetaOpenFOAM: an LLM-based multi-agent framework for CFD (2024).
- Laughman, C. R., A Comparison of Transient Heat-Pump Cycle Simulations with Homogeneous and Heterogeneous Flow Models.
- Onishi, K., Lessons from toyota for building durable multi-agent copilots, [Presented at Microsoft Ignite 2024](#) (2024), Session BRK117.
- P, Ey, S., Eep, Xu, R., Wang, W. and Chu, X., OpenFOAMGPT: a RAG-Augmented LLM Agent for OpenFOAM-Based Computational Fluid Dynamics, available from <http://arxiv.org/abs/2501.06327>.
- Picard, C., Edwards, K. M., Doris, A. C., Man, B., On, Giannone, G., Alam, M. F. and Ahmed, F., From Concept to Manufacturing: Evaluating Vision-Language Models for Engineering Design (2024).
- Xiang, J., Ye, T., Liu, P., Zhang, Y. and Wang, W., Modigen: a large language model-based workflow for multi-task modelica code generation (2025).
- 日本機械学会, 設計のための 1DCAE 概念と実現技術, 日本機械学会 (2020), p. 256, 初版発行日: 2020 年 11 月 2 日.
- 玉谷基亮, 三菱電機グループの持続的なものづくりを支える AI ソリューション群, [三菱電機技報](#), Vol. 98, No. 9 (2024), pp. 3–6.
- 藤田喜久雄, 設計論: 製品設計からシステムズイノベーションへ, コロナ社 (2023), p. 494, 初版第 1 刷発行日: 2023 年 7 月 7 日.